

Adaptive Advance-Time Control for nFAPI Split under Unpredictable Latency

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1[†]

*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan

[†]Affiliation 2

Email: crg@mail.ntust.edu.tw

Abstract—

Index Terms—nFAPI, O-RAN, Functional Split, Adaptive Control.

I. Introduction

A. Background and Motivation

The evolution of fifth-generation (5G) mobile networks toward open and disaggregated architectures has fundamentally transformed the design of Radio Access Networks (RANs). Initiatives such as Open RAN (O-RAN) and 3GPP-defined functional splits [1] enable network functions to be distributed across heterogeneous platforms, thereby improving deployment flexibility, vendor interoperability, and cost efficiency. However, this architectural shift introduces a critical and often under-addressed challenge: maintaining precise timing coordination across physically separated network entities under non-deterministic transport conditions.

In traditional monolithic base stations, inter-layer interactions are tightly coupled within a single hardware platform, where timing relationships are inherently deterministic. In contrast, functional split architectures replace internal data buses with packet-based transport networks, where latency, jitter, and processing delays become non-deterministic [2]. As a result, synchronization is no longer a purely physical-layer problem, but evolves into a cross-layer system challenge involving transport protocols, scheduling logic, and timing window management.

Although multiple functional split options have been standardized, including 3GPP Option 2, Small Cell Forum (SCF) Option 6 (nFAPI), and O-RAN Option 7.2x, they fundamentally differ in their synchronization models and timing sensitivities. Option 2, which separates the Packet Data Convergence Protocol (PDCP) and Radio Link Control (RLC) layers, operates over the F1 interface and primarily relies on message-based coordination and status reporting. Its timing requirements are relatively relaxed, typically at the millisecond level, and timing misalignment is mitigated through buffering and retransmission mechanisms.

In contrast, Option 6 introduces a tighter coupling between the Medium Access Control (MAC) and Physical (PHY) layers through the nFAPI interface [3]. In this architecture, per-slot scheduling decisions must be

delivered within strict deadlines, requiring slot-level synchronization between distributed entities. Synchronization is achieved through timestamp-based message exchanges (e.g., P7 Node Sync), enabling delay estimation and timing alignment. However, unlike lower-layer splits, Option 6 operates over general-purpose transport networks (e.g., Ethernet), where latency variations caused by operating system scheduling and network congestion can significantly disrupt timing determinism.

Option 7.2x further pushes the limits of synchronization by splitting the PHY layer into high-PHY and low-PHY components and transporting time-sensitive in-phase and quadrature (IQ) samples over the fronthaul [4]. This architecture requires nanosecond-level time and phase synchronization, typically achieved through hardware-assisted mechanisms such as IEEE 1588 Precision Time Protocol (PTP) and Synchronous Ethernet (SyncE) [5]. To preserve real-time processing constraints, strict timing windows (e.g., T_{2a} and T_{a4}) are enforced, and packets arriving outside these windows are typically discarded.

These distinct synchronization paradigms also lead to fundamentally different timing window behaviors. In Option 7.2x, timing windows are rigid and enforced at the physical layer, where out-of-window packets are dropped to ensure real-time signal processing. In Option 2, timing misalignment is largely absorbed by higher-layer retransmission and buffering mechanisms, with minimal strict enforcement. Option 6, however, occupies a unique intermediate position: it requires strict slot-level timing alignment but operates over non-deterministic transport networks without hardware-enforced synchronization. Consequently, timing violations often manifest as scheduling failures (e.g., missing subframes), and existing implementations typically rely on static timing configurations that are insufficient under dynamic network conditions.

The rationale for deploying the MAC layer in a cloud environment is primarily driven by resource pooling. Research demonstrates that while physical layer processing load is largely determined by bandwidth and antenna configurations, Layer 2 processing complexity—particularly the MAC scheduler—scales linearly or quadratically with the number of active User Equipments (UEs) [6], [7]. By migrating the MAC layer to a VNF, operators can dynam-

ically allocate computational resources to accommodate fluctuating UE densities [8]. However, this flexibility introduces endogenous latency: as UE count increases, the MAC processing time inevitably lengthens, compressing the available margin for fronthaul transmission delay. Furthermore, operating in non-ideal transport environments with elevated latency disrupts traditional Hybrid ARQ (HARQ) timelines, an issue highlighted in the O-RAN specifications [9]. While Option 6 inherently tolerates higher latency by separating the MAC and PHY layers—making it suitable for low-mobility small cell deployments without requiring expensive precise synchronization hardware—it still demands adaptive synchronization to manage the variable processing and transport delays.

This gap highlights a fundamental limitation in current disaggregated RAN designs: while synchronization mechanisms exist at both extremes (message-level and hardware-level), there is a lack of adaptive timing control mechanisms for intermediate-layer splits operating under non-ideal transport environments.

B. Related Works

The evolution of Radio Access Network (RAN) architectures from Distributed RAN (D-RAN) to Open RAN (O-RAN) requires a thorough study of functional splits [2], [10], [11]. The MAC-PHY split (Option 6), standardized by the Small Cell Forum (SCF) as nFAPI [3], [12], is suitable for small cell deployments. It allows the MAC layer to be virtualized on commercial servers while the PHY layer remains on dedicated hardware.

The performance of the nFAPI split depends on the transport network. Practical deployments often use non-ideal fronthaul, such as Shared Ethernet or DOCSIS. Mehran and MacKenzie [13] demonstrated that uplink throughput drops to zero with a small increase in fronthaul delay. Furthermore, server processing delay is unpredictable. Khalaf et al. [14] indicated that server delay increases under high CPU utilization, causing the MAC scheduler to miss its timing window.

To address fronthaul latency, Huang and Zhang [15] proposed a distributed MAC scheduling scheme. While this scheme reduces the impact of delayed grants, it introduces a “split-brain” issue with multiple schedulers. Akgun et al. [16] utilized intelligent scheduling to adapt modulation based on interference, but did not solve the sub-millisecond timing of message delivery itself. Existing OpenAirInterface (OAI) reference models typically rely on static timing windows, leading to frequent “missing subframe” errors when network jitter exceeds configured limits. This paper bridges this gap by proposing a centralized Adaptive Advance-Time Control algorithm to dynamically manage scheduling timing.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet

transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

We identify that the root cause of performance degradation in such systems lies in the inability of the MAC scheduler to adapt its transmission timing to dynamic end-to-end latency variations. Existing open-source implementations, such as OpenAirInterface (OAI), typically employ static timing advance configurations, which fail to accommodate fluctuating network conditions, leading to frequent synchronization failures and throughput degradation.

To address this issue, we propose an adaptive timing control framework that dynamically adjusts scheduling lead time based on real-time latency estimation. The proposed approach introduces a dual-loop architecture, where a convergence optimization loop estimates end-to-end delay characteristics, and a timing actuator dynamically compensates for timing offsets to ensure that scheduling messages arrive within the valid timing window.

The main contributions of this paper are summarized as follows:

- **Systematic Analysis of Timing Uncertainty in Split 6:** We characterize the impact of non-deterministic latency, including network jitter and processing delay, on synchronization stability in disaggregated RAN systems.
- **Adaptive Advance-Time Control Algorithm:** We propose a dynamic timing adjustment mechanism that continuously aligns scheduling decisions with varying transport delays, effectively mitigating timing violations.
- **Experimental Validation on OAI Platform:** We implement the proposed framework in a real-world testbed and demonstrate that it eliminates missing subframe errors and significantly improves throughput stability under non-ideal fronthaul conditions.

II. System Model

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a resilient two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegasus O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user

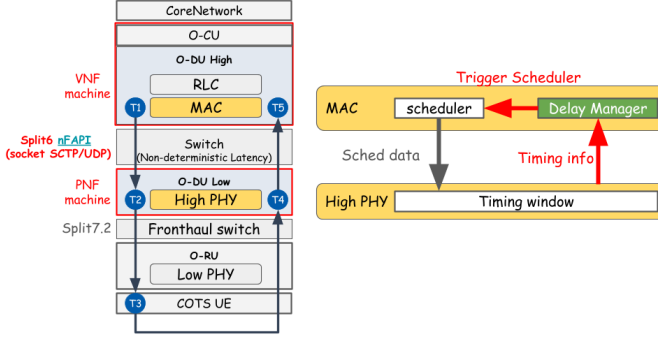


Fig. 1. System architecture and delay management mechanism.

equipment:

- Core Network (CN): The central element responsible for routing, session management, and authenticating user data.
- O-RAN Central Unit (O-CU): A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- O-RAN Distributed Unit (O-DU): To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- Switch: A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- O-RAN Radio Unit (O-RU): A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- Commercial Off-The-Shelf User Equipment (COTS UE): Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC and Delay Manager

Within the Media Access Control (MAC) layer of the O-DU High, the core components handling timing mechanisms include the Scheduler and the nFAPI Delay Manager:

- Scheduler: Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.

- nFAPI Delay Manager: This resides as the crucial control unit of this delay-aware architecture. Because unpredictable, unstable latency exists in the switch between the VNF and PNF, the Delay Manager continuously receives timing feedback from the lower physical layer. It uses this to dynamically calculate network jitter and proactively trigger the scheduler. This calculation guarantees that the scheduling instructions (Sched data) are generated ahead of time to account for network delays.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- Timing Window: A predefined, strict time duration. When the scheduling message (specifically the DL_TTI.request) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the subframe data.
- Timing Info: The PNF continuously monitors the exact arrival timestamp of these control messages and periodically sends this “Timing Info” back to the Delay Manager in the MAC layer. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following Key Validation Metrics:

- MAC Layer HARQ RTT ($T_5 - T_1$): Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- VNF-PNF DL Latency ($T_2 - T_1$): Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- O-RAN Split 7.2x Interoperability: While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [17], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [18], our system bridges these interfaces.
- Hybrid System Architecture for E2E Validation: To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. Proposed Method

This section details the proposed dynamic timing adjustment algorithm. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose a Dual-Loop Timing Control Architecture.

A. Dual-Loop Architecture

The timing control of this system adopts a “policy-execution separation” dual-loop architecture, ensuring that the timing advancement on the VNF side can flexibly cope with network jitter without inducing unexpected oscillations:

- 1) Convergence Optimization Loop (Policy): Dynamically analyzes the statistical data reported by the PNF through the convergence optimization algorithm to determine the global target slot-ahead S_{ahead} .
- 2) Timing Actuator Loop (Execution): Executed by an independent VNF timing thread. This thread is the only unit in the system capable of altering the absolute sleep time and slot propagation, translating the upper-layer policy target and low-layer microsecond phase deviations into actual timing advancement.

B. VNF Timing Actuator

The core function of the VNF Timing Actuator is isolation and precise phase execution. Let T_{slot} denote the slot duration. The target slot-ahead value, S_{next} , is determined by the policy loop. For microsecond-level phase correction, the actuator calculates the next absolute wake-up time, operating within a mutex-locked environment to mitigate long-term error accumulation. By adjusting the absolute wake-up time, it accurately translates the upper-layer policy target and low-layer microsecond phase deviations into actual timing advancement.

C. Adaptive Node Synchronization

To achieve microsecond-level phase alignment, the system utilizes an IEEE 1588 (PTP) style timestamp exchange. When the VNF receives the `UL_node_sync` message, it calculates the network delay and instantaneous timing offset based on four timestamps (t_1 to t_4). To avoid transient spikes, the offset and jitter are smoothed using an Exponentially Weighted Moving Average (EWMA) model. If the offset exceeds a dynamic threshold derived from the estimated jitter, the system breaks the offset down into a slot adjustment (S_{adj}) and a microsecond-level adjustment (μs_{adj}). These parameters are subsequently consumed by the VNF Timing Actuator to precisely align the local clock. The process is summarized in Algorithm 1.

D. Trigger Timing Control (Convergence Optimization)

To counteract fluctuating network latency and maintain scheduling robustness, we propose a dynamic trigger timing control policy. Upon receiving timing statistics (e.g., the worst-case late arrival t_{arrive}) reported by the PNF, the system incorporates an EWMA model inspired by the Jacobson/Karels TCP RTT algorithm to estimate the mean latency μ_{delay} and jitter deviation dev_{delay} .

Before making tracking adjustments, the algorithm establishes multiple Panic Criteria. A statistical anomaly is detected if the absolute error $|error|$ exceeds $3 \cdot dev_{delay}$, indicating a sudden latency spike. A strict violation occurs if $t_{arrive} \geq T_{slot}$, meaning the packet is late beyond the slot boundary. Either condition triggers a Fast Attack Panic Mode, radically jumping the slot-ahead value S_{next} by at least 2 slots to ensure connection continuity and mitigate catastrophic deadline violations.

Conversely, to maximize low-latency operation in safe zones, a Proactive Latency Reduction mechanism is used. When the latency operates safely below an absolute boundary ($t_{arrive} \leq \text{Safe_Margin}$), a stability counter $count_{stable}$ increments. Once this counter exceeds a dynamically scaled threshold (incorporating a leaky bucket penalty that slowly decays over time), the system cautiously drops S_{next} by 1 slot. This asymmetric response (fast expansion, slow decay) prevents ping-pong oscillation while minimizing round-trip time. The adjustment strategy is condensed in Algorithm 2.

IV. Experimental Results

To validate the hypothesis and the effectiveness of the proposed dynamic trigger scheduler algorithm, we systematically designed multiple experimental scenarios. Section IV-A outlines the general software and hardware environment used across all tests. Subsequent sections present specific scenarios: Scenario I (IV-B) isolates and verifies the problems caused by non-deterministic latency under standard OpenAirInterface implementations, while Scenario II (IV-C) and subsequent experiments evaluate the robustness of our proposed time-advance control under varying levels of congestion.

Algorithm 1 Adaptive Node Synchronization

Require: t_1, t_2, t_3 (Timestamps from PNF UL_node_sync), t_4 (VNF local receive timestamp)

Ensure: S_{adj} (Slot adjustment), μs_{adj} (Microsecond adjustment)

Notation:

$offset$ (Instantaneous timing offset), μ_{offset} (EWMA mean offset)

dev_{offset} (EWMA mean absolute deviation / Jitter)

$Threshold$ (Dynamic synchronization threshold)

α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)

Phase I: Timestamp Processing and Wrap-around Protection

1: $d_1 \leftarrow t_2 - t_1$

2: $d_2 \leftarrow t_4 - t_3$

Phase II: Offset and Dynamic Jitter Estimation

3: $offset \leftarrow (d_1 - d_2)/2$

4: $error \leftarrow offset - \mu_{offset}$

5: $\mu_{offset} \leftarrow \mu_{offset} + \alpha \cdot error$

6: $dev_{offset} \leftarrow dev_{offset} + \beta(|error| - dev_{offset})$

7: $Threshold \leftarrow dev_{offset}$

Phase III: Phase Alignment

8: if $|offset| > Threshold$ then

9: $S_{adj} \leftarrow \lfloor offset / slot_duration \rfloor$

10: $\mu s_{adj} \leftarrow offset \pmod{slot_duration}$

11: Apply S_{adj} and μs_{adj} to adjust local clock

12: else

13: State $\leftarrow$ Synchronized

14: $S_{adj} \leftarrow 0, \mu s_{adj} \leftarrow 0$

15: end if

16: return $S_{adj}, \mu s_{adj}$

TABLE I
Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

A. Experimental Scenario

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

B. Scenario I: Analysis of Non-Deterministic Latency

Deploying the VNF and PNF on different machines introduces significant challenges. Because the MAC scheduler is separated from the PHY layer, strict slot synchronization is required between these distributed components. However, even when deployed in the exact same network environment, different traffic loads directly impact the latency. Figure 2 illustrates the latency between the VNF scheduler and the PNF under varying traffic loads. As traffic increases, three factors heighten the overall delay: the scheduler requires more processing time, the data packing time lengthens, and the VNF-to-PNF One-Way Delay (OWD) is affected. Consequently, the VNF cannot reliably transmit data exactly at the slot boundary. Instead, it must dynamically advance its scheduling point to compensate for these varying delays and ensure packets arrive at the PNF on time.

C. Scenario II: [Placeholder for Future Experiment]

References

- [1] 3GPP, “Ng-ran; architecture description,” 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 38.401, 2024, version 18.0.0.
- [2] L. M. P. Larsen, A. Checko, and H. L. Christiansen, “A survey of the functional splits in 5g next-generation radio access networks,” IEEE Communications Surveys & Tutorials, vol. 21, no. 3, pp. 2349–2372, 2019.

Algorithm 2 Trigger Timing Control

Require: t_{arrive} (Timing info from PNF)

Ensure: S_{next} (Trigger timing for the next MAC scheduling)

Notation:

S_{curr} (Current trigger timing), μ_{delay} (EWMA mean delay)

dev_{delay} (EWMA mean deviation)

$count_{stable}$ (Consecutive stable arrivals counter)

N_{stable} (Adaptive stability threshold, dynamically scaled by dev_{delay})

α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)

Phase I: Delay Statistics Update

1: $error \leftarrow t_{arrive} - \mu_{delay}$

2: $\mu_{delay} \leftarrow \mu_{delay} + \alpha \cdot error$

3: $dev_{delay} \leftarrow dev_{delay} + \beta(|error| - dev_{delay})$

Phase II: Anomaly Detection

4: $IsAnomaly \leftarrow (|error| > 3 \cdot dev_{delay}) \vee (t_{arrive} > slot_duration)$

Phase III: Timing Adjustment

5: if $IsAnomaly$ then

6: $S_{next} \leftarrow S_{curr} + 1$

7: $count_{stable} \leftarrow 0$

8: else if $t_{arrive} \leq Safe_Margin$ then

9: $count_{stable} \leftarrow count_{stable} + 1$

10: if $count_{stable} \geq N_{stable}$ then

11: $S_{next} \leftarrow S_{curr} - 1$

12: $count_{stable} \leftarrow 0$

13: else

14: $S_{next} \leftarrow S_{curr}$

15: end if

16: else

17: $S_{next} \leftarrow S_{curr}$

18: $count_{stable} \leftarrow 0$

19: end if

20: Limit S_{next} within the allowable nFAPI timing window

21: return S_{next}

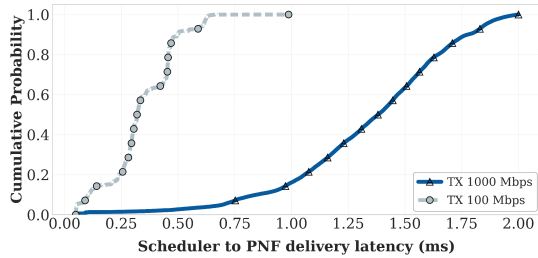


Fig. 2. Observation of latency between VNF scheduler and PNF under different traffic loads.

- [3] Small Cell Forum, “5g nfdpi specification,” Small Cell Forum (SCF), Tech. Rep. SCF222, 2023. [Online]. Available: <https://www.smallcellforum.org/>
- [4] O-RAN Alliance, “O-ran fronthaul working group control, user and synchronization plane specification,” O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v11.00, 2023.
- [5] IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems, IEEE Std. IEEE 1588-2019, 2019.
- [6] X. Foukas et al., “Computational resource consumption of 5g virtualized ran,” in 2021 IEEE International Conference on

Communications (ICC). IEEE, 2021.

- [7] V. Authors, “On the complexity of proportional fair scheduling in 5g new radio,” in 2019 IEEE Wireless Communications and Networking Conference (WCNC). IEEE, 2019.
- [8] —, “An analysis of gpp-based main processors for 5g cloud-ran,” in 2018 IEEE Conference on Standards for Communications and Networking (CSCN). IEEE, 2018.
- [9] O-RAN Alliance, “O-ran fronthaul working group control, user and synchronization plane specification,” O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v02.00, Aug 2019.
- [10] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, “A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond,” IEEE Communications Surveys & Tutorials, vol. 25, no. 4, pp. 2791–2832, 2023.
- [11] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. D. Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, “A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges,” IEEE Communications Surveys & Tutorials, vol. 28, pp. 1637–1678, 2026.
- [12] S. C. Forum, “5g network fapi (nfapi) specification,” Small Cell Forum, Tech. Rep. SCF225, 2020.
- [13] F. Mehran and R. MacKenzie, “Experimental evaluation of multi-vendor virtualized ran using non-ideal fronthaul,” in 2020 IEEE 31st Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC). IEEE,

2020.

- [14] B. N. Khalaf, W. H. Ali, R. S. Alhumaima, and H. A. J. Alshamary, "Delay aware resource allocation in oran through network optimization," IET Wireless Sensor Systems, 2024.
- [15] M. Huang and X. Zhang, "Distributed mac scheduling scheme for c-ran with non-ideal fronthaul in 5g networks," in Communication Infrastructure Research Lab (Intel Labs China). IEEE, 2026.
- [16] B. Akgun et al., "Interference-aware intelligent scheduling for virtualized private 5g networks," IEEE Access, vol. 12, 2024.
- [17] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," IEEE Communications Standards Magazine, vol. 2, no. 3, pp. 32–40, 2018.
- [18] X. Wang et al., "An open-source implementation of the 5g nr functional split option 6," in 2022 IEEE International Conference on Communications (ICC), 2022, pp. 1–6.